PRODUCT & ENGINEERING SPECIFICATION — MVP 1.0

TradeGuard: Zero-API Pre-Trade Checklist Extension

End-to-End System Architecture, Functional Requirements, Complete Manifest V3 Source Code & Google AI Studio Workflow

Product Type	Chrome Extension (Manifest V3)	**Architecture**	100% Client-Side DOM Interception
External APIs	None Required (Zero Broker Keys)	**Target Platforms**	TradingView, Dhan, Zerodha Kite Web
Build Workflow	Google AI Studio + Chrome Local	**Release Scope**	MVP1 Core Gatekeeper & Custom Rules

1. Product Vision & Psychological Thesis

Retail traders rarely fail due to a lack of chart-reading knowledge; over 90% of preventable losses stem from emotional impulse—revenge trading after a loss, chasing green candles out of FOMO, or ignoring position-sizing rules. Traditional trading journals analyze mistakes *after* capital is lost. TradeGuard acts as an uncompromisable, real-time behavioral seatbelt right inside the browser before an order reaches the exchange.

By operating entirely client-side through browser DOM reading and event interception, TradeGuard requires **zero broker API keys, zero login credentials, and zero server round-trips**. This eliminates developer API approval friction, guarantees sub-millisecond response times, and ensures complete trader privacy.

> **Core Value Proposition:** "100% Local Execution — No broker login, no API keys, and zero trading data ever leaves your browser. An instant pre-trade discipline gatekeeper that forces rational verification before every Buy or Sell click."

2. MVP1 Requirements Specification

MVP1 is strictly scoped to validate one hypothesis: reliably intercepting an order-submission click on a web trading terminal, blocking execution with a dark-mode checklist overlay, and unlocking the trade only after all user-defined rules are verified.

Req ID	Feature Module	Functional Specification	Priority
FR-01	DOM Click Interceptor	Attach a capture-phase event listener (`useCapture = true`) to intercept clicks on Buy, Sell, and Place Order buttons before the broker's frontend handlers execute.	Must-Have

Req ID	Feature Module	Functional Specification	Priority
FR-02	Screen Context Reader	Read visible order ticket parameters directly from the DOM (e.g., button label Buy vs. Sell, active ticker symbol, or input quantity) to display context inside the modal.	Must-Have
FR-03	Gatekeeper Modal Overlay	Inject an isolated Shadow DOM dark-mode overlay centered on screen that dims the broker interface and disables the final confirm button until all checkboxes are ticked.	Must-Have
FR-04	Keyboard Rapid-Check	Support numeric keys (`1` – `9`) to toggle checklist items and `Enter` to confirm once unlocked, preventing execution slippage for active day traders.	Must-Have
FR-05	Custom Rule Manager	Provide an extension popup UI allowing traders to add, delete, and persist up to 5 custom trading rules in `chrome.storage.local`, plus a master On/Off toggle.	Must-Have
FR-06	Safe Replay Mechanism	When the trader completes the checklist and clicks "Confirm & Place Order," temporarily bypass the interceptor flag and re-dispatch the original click event cleanly.	Must-Have
NFR-01	Zero-API & Privacy	No external network requests (`fetch` / `XHR`), no remote scripts, and no broker API credentials stored or requested anywhere in the codebase.	Critical

3. Client-Side Architecture & Execution Flow

TradeGuard uses Chrome's Manifest V3 architecture with three decoupled local components communicating through native browser APIs and DOM events. Style isolation is enforced via a `ShadowRoot` so broker CSS stylesheets cannot distort the checklist modal.

Stage	Component	Execution Logic & Data Flow
Step 1	Extension Popup (`popup.html/js`)	User configures custom trading rules (e.g., *"Stop loss placed?"*, *"Risk < 1%?"*) and toggles the guard active. Data is saved to `chrome.storage.local`.
Step 2	Capture Listener (`content.js`)	Injected into trading tabs at `document_start`. Listens on `document` in the capture phase for clicks matching order-button selectors or text patterns (`Buy` , `Sell` , `Place Order`).
Step 3	Event Freeze & DOM Scrape	Calls `e.preventDefault()` , `e.stopPropagation()` , and `e.stopImmediatePropagation()` . Scrapes visible order side (Buy/Sell) and symbol text from nearby DOM nodes.

Stage	Component	Execution Logic & Data Flow
Step 4	Shadow DOM Modal	Renders the dark-mode checklist overlay with scraped order context. Keeps the submit button disabled until every rule checkbox is checked.
Step 5	Verified Pass-Through	Upon user confirmation, sets `isBypassing = true`, removes the modal, programmatically triggers `targetButton.click()`, and resets the bypass flag on the next tick.

Project Directory Structure

```
tradeguard-mvp1/
├── manifest.json       # Manifest V3 permissions, host matches, and script registration
├── content.js          # Capture-phase click interceptor, DOM context reader & Shadow DOM
modal
├── popup.html          # Extension toolbar UI for toggling guard & managing custom rules
└── popup.js            # Local storage persistence logic for user-defined checklist rules
```

4. Complete MVP1 Source Code (Copy-Paste Ready)

Create a folder named `tradeguard-mvp1` on your computer and save the four files below inside it. All styling is encapsulated inside JavaScript and HTML with zero external dependencies.

File 1: manifest.json

```
{
  "manifest_version": 3,
  "name": "TradeGuard — Pre-Trade Discipline Checklist",
  "version": "1.0.0",
  "description": "Zero-API pre-trade checklist gatekeeper that prevents emotional impulse
orders.",
  "permissions": ["storage"],
  "action": {
    "default_popup": "popup.html",
    "default_title": "TradeGuard Rules"
  },
  "content_scripts": [
    {
      "matches": [
        "https://*.tradingview.com/*",
        "https://*.dhan.co/*",
        "https://kite.zerodha.com/*"
      ],
      "js": ["content.js"],
      "run_at": "document_start"
    }
  ]
}
```

File 2: popup.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <style>
    body { width: 320px; margin: 0; padding: 14px; background: #0f172a; color: #f8fafc;
font-family: system-ui, sans-serif; font-size: 13px; }
    .header { display: flex; justify-content: space-between; align-items: center; border-
bottom: 1px solid #1e293b; padding-bottom: 10px; margin-bottom: 12px; }
    h2 { margin: 0; font-size: 15px; color: #38bdf8; }
    .row { display: flex; gap: 6px; margin-bottom: 10px; }
    input[type="text"] { flex: 1; padding: 7px 9px; border-radius: 4px; border: 1px solid
#334155; background: #1e293b; color: #fff; outline: none; }
    button { padding: 7px 12px; border: none; border-radius: 4px; background: #0284c7;
color: #fff; font-weight: 600; cursor: pointer; }
    button:hover { background: #0369a1; }
    ul { list-style: none; padding: 0; margin: 0; max-height: 220px; overflow-y: auto; }
    li { display: flex; justify-content: space-between; align-items: center; background:
#1e293b; padding: 7px 9px; border-radius: 4px; margin-bottom: 6px; border: 1px solid
#334155; }
    .del-btn { background: transparent; color: #f87171; padding: 2px 6px; font-size: 14px; }
    .status { font-size: 11px; color: #94a3b8; margin-top: 8px; }
  </style>
</head>
<body>
  <div class="header">
    <h2>TradeGuard MVP1</h2>
    <label><input type="checkbox" id="guardEnabled" checked> Active</label>
  </div>
  <div class="row">
    <input type="text" id="newRuleInput" placeholder="Add pre-trade rule (max 5)..."
maxlength="90">
    <button id="addRuleBtn">Add</button>
  </div>
  <ul id="rulesList"></ul>
  <div class="status">100% Local Storage • Zero API Keys Required</div>
  <script src="popup.js"></script>
</body>
</html>
```

File 3: popup.js

```
const DEFAULT_RULES = [
  "Is my Stop-Loss strictly defined and entered?",
  "Am I risking less than 1% of total capital?",
  "Is this setup aligned with the higher-timeframe trend (No FOMO)?"
];

const guardEnabledEl = document.getElementById("guardEnabled");
const newRuleInput = document.getElementById("newRuleInput");
const addRuleBtn = document.getElementById("addRuleBtn");
const rulesList = document.getElementById("rulesList");

function renderRules(rules) {
  rulesList.innerHTML = "";
  rules.forEach((rule, index) => {
    const li = document.createElement("li");
    const span = document.createElement("span");
    span.textContent = `${index + 1}. ${rule}`;
    const delBtn = document.createElement("button");
    delBtn.textContent = "×";
    delBtn.className = "del-btn";
    delBtn.onclick = () => {
      const updated = rules.filter((_, i) => i !== index);
      chrome.storage.local.set({ rules: updated }, () => renderRules(updated));
    };
    li.append(span, delBtn);
    rulesList.appendChild(li);
  });
}

chrome.storage.local.get({ rules: DEFAULT_RULES, enabled: true }, (data) => {
  guardEnabledEl.checked = data.enabled;
  renderRules(data.rules);
});

guardEnabledEl.addEventListener("change", () => {
  chrome.storage.local.set({ enabled: guardEnabledEl.checked });
});

addRuleBtn.addEventListener("click", () => {
  const text = newRuleInput.value.trim();
  if (!text) return;
  chrome.storage.local.get({ rules: DEFAULT_RULES }, (data) => {
    if (data.rules.length >= 5) return alert("MVP1 supports up to 5 active rules.");
    const updated = [...data.rules, text];
    chrome.storage.local.set({ rules: updated }, () => {
      newRuleInput.value = "";
      renderRules(updated);
    });
  });
});
```

File 4: content.js

```
(function () {
  const DEFAULT_RULES = [
    "Is my Stop-Loss strictly defined and entered?",
    "Am I risking less than 1% of total capital?",
    "Is this setup aligned with the higher-timeframe trend (No FOMO)?"
  ];

  let isBypassing = false;
  let activeModalHost = null;

  function isOrderSubmitButton(el) {
    if (!el) return null;
    const btn = el.closest("button, [role='button'], input[type='submit']");
    if (!btn || btn.disabled) return null;

    const label = (btn.innerText || btn.value || btn.getAttribute("aria-label") ||
"").trim().toLowerCase();
    const orderKeywords = ["buy", "sell", "place order", "submit order", "buy market", "sell
market"];
    const matchesKeyword = orderKeywords.some((kw) => label === kw || label.startsWith(kw +
" "));
    const matchesDataAttr = btn.matches("[data-name='submit-order'], .order-buy-button,
.order-sell-button");

    return matchesKeyword || matchesDataAttr ? btn : null;
  }

  function extractScreenContext(targetBtn) {
    const actionText = (targetBtn.innerText || targetBtn.value ||
"ORDER").trim().toUpperCase();
    const side = actionText.includes("SELL") ? "SELL" : actionText.includes("BUY") ? "BUY" :
"ORDER";
    const titleParts = document.title.split(" ");
    const symbolGuess = titleParts[0] ? titleParts[0].replace(/[^A-Z0-9:_-]/gi, "") :
"ACTIVE CHART";
    return { side, symbol: symbolGuess || "ACTIVE CHART" };
  }

  function openChecklistModal(targetBtn, rules) {
    if (activeModalHost) activeModalHost.remove();

    const context = extractScreenContext(targetBtn);
    const host = document.createElement("div");
    host.id = "tradeguard-shadow-host";
    const shadow = host.attachShadow({ mode: "open" });

    const sideColor = context.side === "SELL" ? "#ef4444" : "#10b981";

    shadow.innerHTML = `
      <style>
        .backdrop { position: fixed; inset: 0; background: rgba(2, 6, 23, 0.82); z-index:
2147483647; display: flex; align-items: center; justify-content: center; font-family:
system-ui, -apple-system, sans-serif; }
        .card { width: 420px; max-width: 92vw; background: #0f172a; border: 1px solid
#334155; border-radius: 10px; padding: 20px; color: #f8fafc; }
        .top { display: flex; justify-content: space-between; align-items: center; margin-
bottom: 12px; border-bottom: 1px solid #1e293b; padding-bottom: 10px; }
        .badge { background: ${sideColor}; color: #fff; font-weight: 700; font-size: 11px;
padding: 3px 8px; border-radius: 4px; }
        h3 { margin: 0; font-size: 16px; color: #38bdf8; }
        p.sub { margin: 0 0 14px 0; font-size: 12.5px; color: #94a3b8; }
```

5. Google AI Studio Build & Browser Testing Guide

Because you are building directly in **Google AI Studio** in your web browser without installing a local desktop IDE, follow this 4-step workflow to customize, load, and test MVP1:

1. **Create the Local Folder:** On your computer, create a folder named `tradeguard-mvp1` and save the four files (`manifest.json`, `popup.html`, `popup.js`, `content.js`) inside it.
2. **Load Unpacked into Google Chrome:** Open a new browser tab, go to `chrome://extensions/`, enable **Developer mode** in the top-right corner, click **Load unpacked**, and select your `tradeguard-mvp1` folder.
3. **Test on a Paper Trading Terminal:** Open TradingView Paper Trading (or your broker web terminal), click any **Buy** or **Sell** button, and verify that TradeGuard intercepts the click and displays your checklist modal. Press keys `1`, `2`, `3` and hit `Enter` to test rapid keyboard unlocking.
4. **Iterate via Google AI Studio:** Whenever you want to add broker-specific DOM selectors (for example, scraping exact stop-loss price or lot quantity fields from Dhan or Kite Web), paste your `content.js` into Google AI Studio along with the HTML snippet of your broker's order ticket, ask Gemini to update the selector logic, replace the local file, and click the **Reload** icon in `chrome://extensions/`.